

Demo Video Script & Talking Points

TAKE-HOME ASSIGNMENT

Banking CRM — Conversation-based Agentic AI · Author: Ayush Rawat · Target length: 7–9 minutes

1. Before You Record

1. App running: `npm run dev` → `http://localhost:5173`

2. Login: `rm@bank.com` / `password123`

3. Optional: Swagger open at `http://localhost:3000/api/docs`

4. Browser zoom ~110%, hide bookmarks bar

5. Target length: **7–9 minutes** (assignment asks 5–10)

2. Video Structure

Time	Section	Goal
0:00–0:45	Intro + problem	Show you understand the use case
0:45–2:00	Architecture	System design score
2:00–6:30	3 live demos	Output quality + agentic thinking
6:30–8:00	Trade-offs + design	Clarity / evaluation criteria
8:00–8:30	Wrap-up	Clean close

3. Full Script

3.1 Intro (0:00–0:45)

ON SCREEN: LOGIN → DASHBOARD

“Hi, I’m Ayush. This is my take-home: a conversation-based Agentic AI for Banking CRM. The problem: a Relationship Manager asks in natural language — find high-value customers likely to convert for a personal loan and generate personalized WhatsApp messages. My system does that end-to-end: retrieve CRM data, score conversion likelihood with explainable rules, recommend products, draft WhatsApp messages, and require human approval before anything is sent.”

3.2 Architecture (0:45–2:00)

ON SCREEN: README ARCHITECTURE DIAGRAM OR PACKAGES/ + DOCS

“Architecture is a monorepo: Frontend React + Vite RM portal; Backend NestJS REST API; Agent core LangGraph multi-agent workflow; Engines for scoring, recommendation, prompts, and evaluation; Data in PostgreSQL with 1,000 seeded customers, Redis, and Chroma configured. Flow: RM query → Planner → parallel retrieval tools → Scoring → Recommendation → Messaging → Audit → Response. Agents share AgentState; they don’t call each other directly. Tools talk to the DB. LLM is used mainly for planning and message generation — scores stay deterministic so they’re explainable.”

Mention:

- Clear task decomposition (Planner → Retrieval → Score → Rec → Message)
- Tool/API usage (Customer, Transaction, CRM, Loan, Campaign, Scoring, Recommendation)
- State/context via LangGraph + conversation persistence
- Modular packages (extensible)

3.3 Demo 1 — Personal Loan (2:00–3:30)

ON SCREEN: AI AGENT TAB · CORE ASSIGNMENT USE CASE

Query: *Find high-value customers likely to convert for a personal loan this month and generate personalized WhatsApp messages.*

“Demo 1 — the core use case.” [Run workflow] “You can see a compact execution pipeline: Plan → Data → Score → Match → Messages. Details open on View steps — not a huge timeline.” “Each row has conversion score, confidence, product, and reasons. Scores come from business rules — income, credit, relationship, risk — not a black-box LLM number. Messages use customer context: name, product fit, language.” “Messages are DRAFT only — human-in-the-loop.”

3.4 Demo 2 — Credit Card (3:30–4:45)

ON SCREEN: AI AGENT · DIFFERENT PRODUCT PATH

Query: *Identify customers eligible for premium credit cards with strong transaction history.*

“Demo 2 — different product intent. Planner detects credit card intent. Retrieval and eligibility rules change. Same orchestration, different product path — shows the design is extensible, not hardcoded for one product.” [Show recommendations sorted by confidence then score]

3.5 Demo 3 — Approvals + CRM (4:45–6:30)

ON SCREEN: APPROVALS → CUSTOMERS → DASHBOARD

“Demo 3 — RM operations. Generated messages land here for Approve / Edit / Reject. No auto-send. I’ll approve one, edit one, reject one.” “Full CRM browse — search, profile with transactions, loans, notes — so the agent isn’t a black box; RMs can verify the customer.” “Analytics: pending approvals, product mix, pipeline health.”

Optional if time: Recommendations page (sorted by confidence/score) or Fixed Deposit sample query.

3.6 Design Decisions & Trade-offs (6:30–8:00)

“Key decisions and trade-offs: 1. Deterministic scoring, LLM for language — numbers stay auditable; LLM personalizes copy. 2. Human approval — banking safety over full automation. 3. LangGraph — shared state, retries, feature flags (memory, audit, streaming). 4. Monorepo packages — scoring/recommendation/prompts can evolve independently. 5. Trade-offs: scoring is heuristic, not trained ML; WhatsApp is draft-only; Chroma is wired for future semantic search; seed is 1k customers for demo scale. 6. Evaluation package — business/security/agent evaluators for quality checks.”

3.7 Close (8:00–8:30)

“Deliverables: GitHub repo with modular agents/tools/engines, README with architecture and setup, and this demo of three use cases plus trade-offs. Thanks.”

4. Evaluation Checklist — What to Mention

System Design

- Monorepo: `apps/` + `packages/` (agent-core, scoring-engine, recommendation-engine, prompts, evaluation-engine)
- NestJS orchestrator + LangGraph workflow
- Prisma schema (~16–17 models), seed volumes
- Feature flags: memory, audit, streaming, cache

Agentic Thinking

- Natural language → Planner intent + product type
- Parallel retrieval (customers, transactions, CRM, loans, campaigns)
- Score → eligibility → recommend → message → audit
- Shared `AgentState`, not ad-hoc agent-to-agent calls

Tool Usage

Tool	Role
CustomerTool	High-value candidates
TransactionTool	Spend / balance signals
CrmTool	Notes / intent
LoanTool	Existing credit
CampaignTool	Prior outreach
ScoringTool	Conversion score 0–100
RecommendationTool	Product fit + rank

Tool	Role
MessageGeneration	Personalized WhatsApp drafts

Output Quality

- Reasons + evidence on recommendations
- Confidence + conversion score
- Personalized message content
- Sorted by confidence, then score

Clarity / Docs

- README: architecture, flow, tools, decisions, trade-offs, setup
- Extra docs in `docs/` (agent, tools, workflow, API, evaluation, business rules)
- Login + run instructions in README

Avoid sounding like disqualifiers

- Say scores are **rule-based**, not invented by the LLM
- Show **tools hitting Postgres**, not hardcoded lists
- Point at README + docs

5. Three Use Cases (Must Show)

1. **Personal loan + WhatsApp** — main assignment query
2. **Premium credit card** — different product path
3. **Approvals workflow** — approve / edit / reject (strongest banking CRM story)

Optional 4th: Fixed deposit query from sample chips.

6. README Fixes Before Submit

1. Project structure: use real package names (`agent-core` , `scoring-engine` , `recommendation-engine` , etc.)
2. API path: confirm `POST /api/v1/agent/query` matches code
3. Align seed counts (1,000 customers, not “50”)

7. Recording Tips

- Narrate **why** (explainability, HITL, modularity), not only clicks
- One reject/edit shows you thought about risk
- Don't dwell on CSS — spend time on agent flow, tools, trade-offs
- If OpenAI key is missing: say fallback messages still work; scoring/tools still run